

ネットワーク及び演習

第 3 回 echo クライアント・サーバ、Python の関数

佐野 彰

本日の到達目標

2 つのプログラムが通信するとはどういうことか

- クライアントとサーバの役割の違いを説明できる
- ポート番号が何を識別しているかを説明できる
- TCP と UDP の違いを、それぞれの使い道とあわせて説明できる
- ソケットを使った送受信の流れ（`connect` / `sendall` / `recv`）をコードで追える

Python の関数

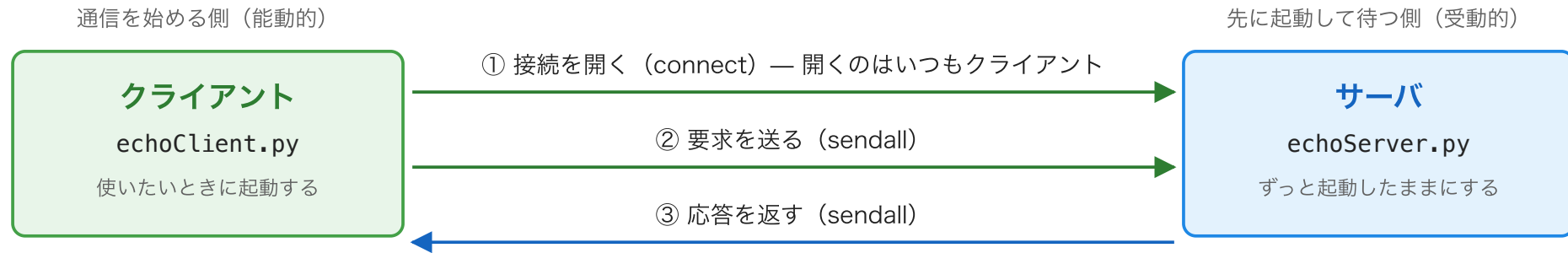
- `def` で関数を定義し、引数と戻り値をやり取りできる

今日から、実際に動くプログラムを読んでいく。`echoClient.py` と `echoServer.py` は 5 講時に自分の PC で動かす。

1. 2つのプログラムが通信する

クライアント・サーバ、ポート、TCP/UDP

クライアント・サーバモデル

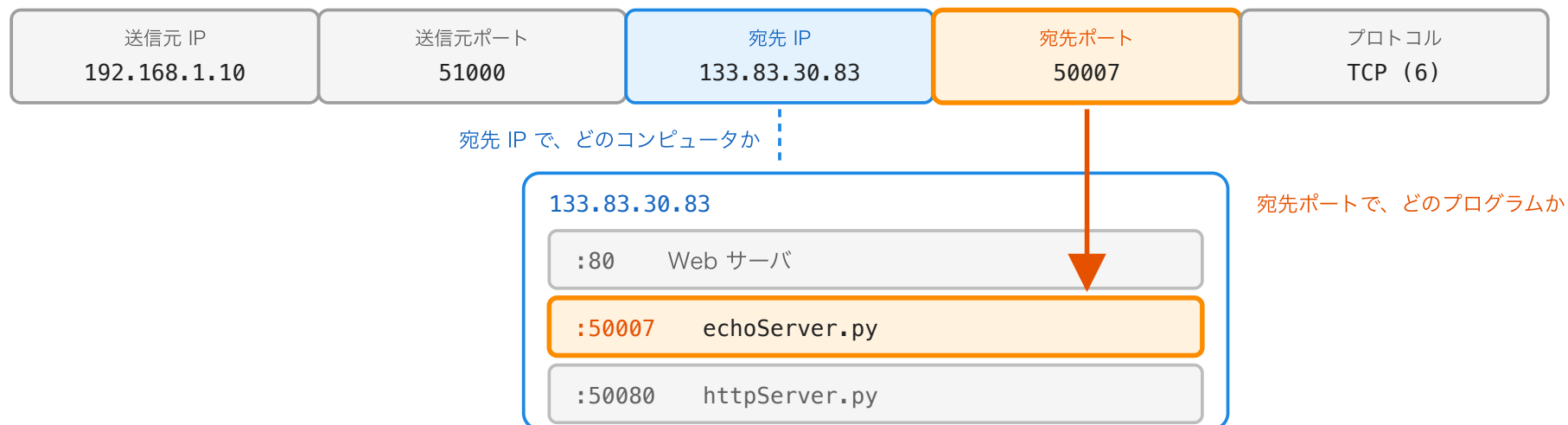


要求 1 回に応答 1 回 — これがこのモデルの約束。だから「いま誰の番か」がいつでも決まる。
送りっぱなしや、回数に対応しない通信（動画配信、通知の push）は、別のモデルで作る。

- サーバはサービスを提供する側。先に起動して、クライアントからの接続を待つ
- クライアントはサービスを使う側。通信は必ずクライアントから始める
- WWW (HTTP)、メール (SMTP)、DNS は、どれもこの形で通信する

ポート番号 — 1 台の計算機の中で通信相手のプログラムを区別する

届いたデータには、この 5 つがいつも付いている (association)



IP アドレスは建物の住所、ポート番号はその中の部屋番号。両方そろって初めて相手が決まる。

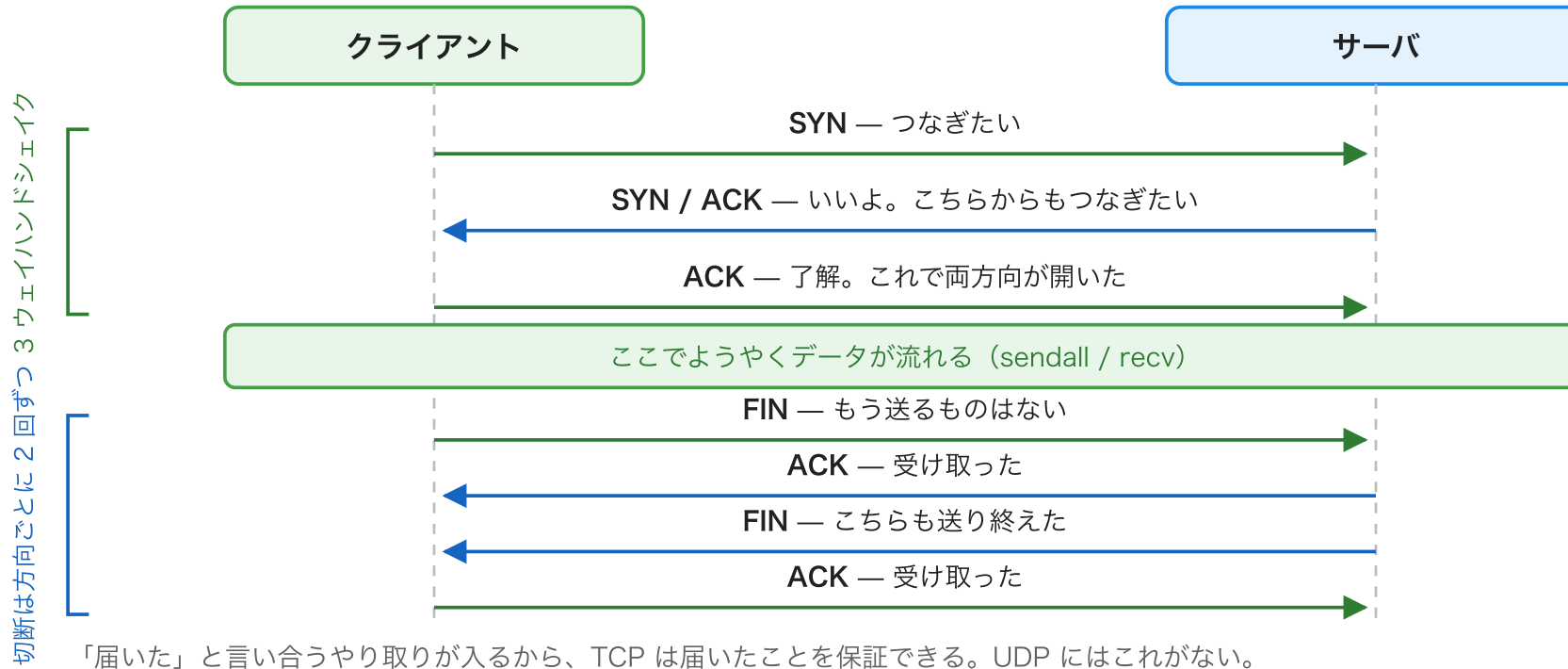
- 0~1023 は **Well-known Port** (HTTP=80, HTTPS=443, SSH=22)。決まったサービス用なので、演習では使わない
 - 対応表は `/etc/services` (macOS・Linux)、`C:\Windows\System32\drivers\etc\services` (Windows)
- 演習では 50007 や 50080 など、空いている大きな番号を使う

TCP と UDP — 到達を確認するかどうか

	TCP (RFC 9293)	UDP (RFC 768)
データの見え方	ストリーム型（切れ目のない流れ）	データグラム型（1 個ずつの小包）
到達の保証	する（届かなければ再送）	しない（送ったら確認しない）
順序の保証	する	しない
速さ	確認のぶん遅い	速い・軽い
使いどころ	Web、メール、ファイル転送	動画・音声、DNS、ゲーム

- 多少欠けても遅れずに届くことを優先する用途では UDP を使う
- この科目で書くのはすべて TCP （ `socket.SOCK_STREAM` ）

TCP は接続を確立してからデータを送る



- 接続を開くのに 3 回、閉じるのに 4 回のやり取りが要る
- `connect()` を 1 行呼ぶと、OS がこのやり取りを行う。つながらないときは、まずこの段階を確認する

2. echo クライアント・サーバ

いちばん単純なプロトコルで、通信の基本的な流れを確認する

echo プロトコル — 受け取ったものをそのまま返す



RFC 862。中身を解釈しないので、経路・ポート・自分のコードのどこが悪いのかを切り分けられる。

- RFC 862。本来は TCP/UDP の 7 番ポート（この科目では 50007 を使う）
- 受け取った中身を解釈しないので、データが相手まで届いているかを確認するのに使える

ソケット — 通信をファイルのように扱う仕組み

ファイル	ソケット
open()	socket() + connect()
write()	sendall()
read()	recv()
close()	close()

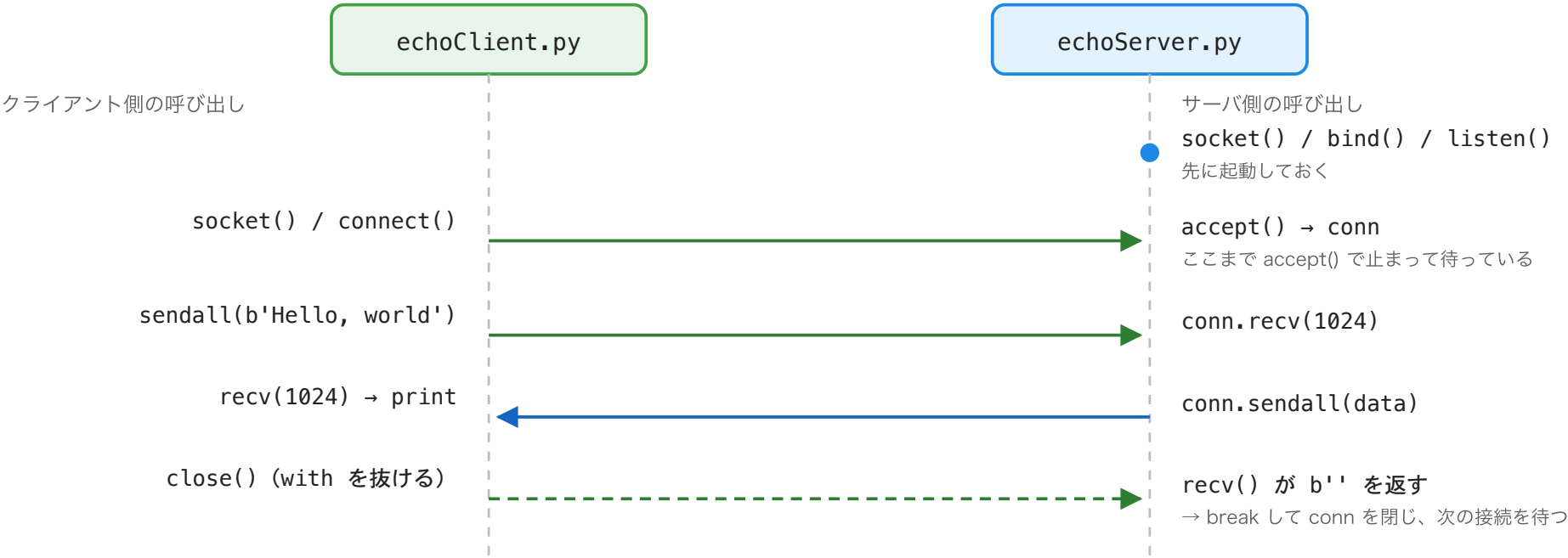
`import socket` で使う。OS が提供するソケットの機能を、Python から呼び出している。

```
import socket

with socket.socket(socket.AF_INET,
                    socket.SOCK_STREAM) as s:
    s.connect(('localhost', 50007))
    s.sendall(b'Hello, world')
    data = s.recv(1024)
    print('Received', repr(data))
```

- `AF_INET` = IPv4、`SOCK_STREAM` = TCP
- 接続先は **タプル** (ホスト名, ポート番号)

どの呼び出しが、どの矢印に対応するか



矢印 1 本ごとに、両側で対になる呼び出しがある。片方だけ書くと、もう片方はそこで待ち続ける。

動かしてみる — ターミナルを 2 つ使う

ターミナル A（先に起動）

```
$ python3 echoServer.py
Connected by ('127.0.0.1', 53578)
Received b'Hello, world'
Send b'Hello, world'
```

- 起動しても何も表示されない。クライアントからの接続を待っている
- `Ctrl-C` で止めるまで、次の接続を待ち続ける

`Connected by` の後ろの数字（53578 など）は、クライアント側のポート番号。OS が接続ごとに空いている番号を割り当てるので、実行するたびに変わる。

ターミナル B（あとから実行）

```
$ python3 echoClient.py
Send b'Hello, world'
Received b'Hello, world'
```

- 文字列を送り、返ってきた文字列を表示して終了する
- なぜこの順に表示されるのかを、`echoClient.py` を上から読んで確かめる

echoClient.py を追う ① — 相手を決めて、つなぐ

```
import socket
HOST = 'localhost'    # 相手のホスト名
PORT = 50007          # 相手のポート番号

with socket.socket(socket.AF_INET,
                   socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
```

- `localhost` は自分の PC を指す。この演習では、サーバとクライアントが同じ PC で動く
- `PORT` はサーバの `PORT` と同じ番号にする。番号が違うと、別のプログラムに接続しようとする
- 電話にたとえると、`socket()` が電話機の用意、`connect()` が電話をかける操作にあたる（TCP の 3 ウェイハンドシェイク）
- 接続すると、ターミナル A に `Connected by (...)` が表示される

サーバが動いていないと、`connect()` の行で `ConnectionRefusedError` になる。

echoClient.py を追う ② — 送る

```
send = b'Hello, world'
s.sendall(send)
print('Send', repr(send))
```

- `b'...'` はバイト列。ネットワークで送れるのはバイト列だけ（詳しくは第 4 回）
- `sendall()` から戻ったあと、次の `print` が `Send b'Hello, world'` を表示する
- サーバはこれを `recv()` で受け取り（A に `Received ...`）、そのまま `sendall()` で送り返す（A に `Send ...`）

echoClient.py を追う ③ — 受け取って、終わる

```
data = s.recv(1024)
print('Received', repr(data))
# ← with を抜けると接続が閉じる
```

- `recv()` は返事が届くまで待つ。届いたデータは `data` に入り、`Received b'Hello, world'` が表示される
- `with` のブロックを抜けると、ソケットが自動で閉じる（`close()` を書かなくてよい）
- サーバは接続が閉じられたことを検出し、次のクライアントを待つ状態に戻る。そのため、クライアントは何度でも実行できる

送る文字列を書き換えても、同じ文字列がそのまま返ってくる。これが echo の動作で、5 講時に確かめる。

3. Python の関数

まとまった処理に名前を付ける

関数の定義と呼び出し

```
def add(a, b):  
    return a + b  
  
result = add(3, 4)  
print(result)      # 7  
  
def greet(name='world'):  
    print(f'Hello, {name}')  
greet()             # Hello, world  
greet('Ryukoku')    # Hello, Ryukoku
```

- `def 関数名(引数, ...):` で定義し、中身はインデントして書く
- `return` で値を返す。`return` がなければ `None` が返る
- 引数にはデフォルト値を指定できる
- 関数を呼び出す前に、その `def` が実行されている必要がある

考え方は C の関数とほぼ同じ。違いは、型を書かないことと、`return` `a, b` のように複数の値を返せること。

なぜ関数にするのか

- 同じ処理を何度も書かずに済む。これは C でも同じ
- 関数名が処理の説明になる。 `httpResponse(conn, addr)` と書けば、中身を読まなくても何をするか分かる
- 処理のまとまりを、別の実行の流れ（スレッド）に渡せる。この科目では、この使い方が重要になる

```
# 第 6 回でこう書く：「この関数を、別のスレッドで動かして」  
http_thread = threading.Thread(target=httpResponse, args=(conn, addr))  
http_thread.start()
```

- 1 つの接続の処理を関数にまとめておくと、複数の接続を同時に処理できる
- 今日の `echoServer.py` は 1 接続ずつ処理する。第 6 回で、ここを複数接続に対応させる

今日やること・次回

5 講時（1-608 で実習）

1. `echoServer.py` を起動し、別のターミナルから `echoClient.py` を動かす
2. `echoClient.py` の送る文字列を書き換えて、そのまま返ってくることを確かめる
3. paiza ラーニング「新・Python入門編」18～20（3 レッスン）の認定証を、まとめて 1 回で aiJudge へ提出
4. aiJudge の **ex3** に取り組む

サーバは起動したままにしておく。サーバを止めると、クライアントの実行は `ConnectionRefusedError` で失敗する。

次回（10/09）：`echoClient.py` と `echoServer.py` を読む

- バイト型（`bytes`）と文字列型（`str`）、`with` 文、`repr()`、サーバ側のソケットが 2 つある理由

質問は Teams「★ 質問用チャネル」か、教室の TA へ。